

AMDP 練習 8：Code-to-Data 實戰改寫

Lecture

前面七題都是「一開始就用 AMDP 寫」的情境。這題反過來：先看一段典型的 **ABAP** 報表邏輯（取數 + 應用層迴圈運算），再把它改寫成 **AMDP** 版本，實際比較兩者的可讀性、正確性、效能。這是這門課從 am01 開始不斷提到的 Code-to-Data 觀念，第一次真的拿實測數字說話。

業務場景沿用 OOP 課程 op12 期末重構的模型（航班營收報表）：查出符合條件的航班，逐筆算 **營收 = 票價 × 已訂位座位數**，可以選擇要不要排除「未售出」（**seatsocc = 0**）的航班。op12 把這段邏輯重構成「取數薄、計算純」的 OO 設計（**get_revenues** 只管 SELECT，**build** 只管純計算），這題則示範另一個維度的重構方向：把「計算」這件事，從 ABAP 應用層搬到 SQLScript 資料庫層。

學習目標

- 能把一段「SELECT + 應用層 LOOP 逐筆運算」的 ABAP 邏輯，改寫成「SELECT 內建運算（**CAST(a * b AS ...)**）」的 SQLScript 版本，理解兩者在邏輯上是等價的
- 學會用 **GET RUN TIME FIELD** 量測一段程式碼的實際執行時間（微秒），這是這門課第一次認真做「能量量測」而不是只看邏輯對不對
- 理解 **Code-to-Data** 不是「無條件更快」：這題實測發現 AMDP 呼叫第一次執行比 ABAP 迴圈版本慢很多（涉及資料庫端的執行計畫編譯），但後續重複呼叫因為執行計畫已經編譯快取，反而比 ABAP 版本快——這代表「要不要下推」要考慮呼叫頻率、資料量，不是看到迴圈就無腦下推
- 知道「何時該下推、何時不該」的判斷原則：純粹的集合運算（篩選、聚合、簡單的逐筆算術）很適合下推；牽涉到呼叫其他 ABAP 邏輯、需要複雜業務規則判斷、或資料量小到迴圈成本可忽略的情境，不一定值得下推

事前準備

ADT 端物件已由課程準備好（\$TMP）：

- **ZCL_AM08_REVENUE_CLASSIC**——改寫前：純 ABAP，SELECT 取數之後用 **LOOP AT ... ASSIGNING** 逐筆計算 **revenue = price * seatsocc**，**iv_skip_unsold = abap_true** 時先 **DELETE** 未售出航班
- **ZCL_AM08_FLIGHT_REVENUE**——改寫後：AMDP 版本，同一段邏輯整段搬進 SQLScript 的 **SELECT**，用 **CAST(price * seatsocc AS DECIMAL)** 直接算出 **revenue**，篩選也用 **WHERE** 子句的布林旗標 **OR**（承 REST rs05 / am02 教過的手法）
- **ZR_AM08_DEMO**——demo 程式，同時呼叫兩個版本，比對筆數、總營收是否一致，並用 **GET RUN TIME FIELD** 量測各自耗時

題目需求（對照已建好的答案物件）

1. **ZCL_AM08_REVENUE_CLASSIC**（改寫前，純 ABAP）：

```
``abap METHOD get_revenues. SELECT f~carrid, c~carrname, f~connid, f~fldate, f~seatsocc, f~price,
f~currency FROM sflight AS f INNER JOIN scarr AS c ON c~carrid = f~carrid WHERE f~carrid = @iv_carrid
ORDER BY f~connid, f~fldate INTO CORRESPONDING FIELDS OF TABLE @rt_revenue.
```

```
IF iv_skip_unsold = abap_true. DELETE rt_revenue WHERE seatsocc = 0. ENDIF.
```

```
LOOP AT rt_revenue ASSIGNING FIELD-SYMBOL(<ls_revenue>). <ls_revenue>-revenue = <ls_revenue>-price *
<ls_revenue>-seatsocc. ENDLOOP. ENDMETHOD. ``
```

典型的「先搬資料、再算」寫法：**SELECT / JOIN** 把整批資料搬到應用層，**DELETE / LOOP** 都是應用層在做。

1. **ZCL_AM08_FLIGHT_REVENUE** (改寫後，AMDP)：

```
``abap METHOD get_revenues BY DATABASE PROCEDURE FOR HDB LANGUAGE SQLSCRIPT OPTIONS READ-
ONLY USING scarr sflight.
```

```
et_revenue = SELECT f.carrid, c.carrname, f.connid, f.fldate, f.seatsocc, f.price, f.currency, CAST(f.price *
f.seatsocc AS DECIMAL(15,2)) AS revenue FROM sflight AS f JOIN scarr AS c ON c.mandt = f.mandt AND
c.carrid = f.carrid WHERE f.mandt = :iv_mandt AND f.carrid = :iv_carrid AND ( f.seatsocc > 0 OR :iv_skip_unsold
<> 'X' ) ORDER BY f.connid, f.fldate;
```

```
ENDMETHOD. ``
```

同一段邏輯，在 SQLScript 裡：JOIN 還是 JOIN，篩選變成 **WHERE** 子句 (**f.seatsocc > 0 OR :iv_skip_unsold <> 'X'**，跟 am02/rs05 教過的布林旗標 OR 是同一招)，計算變成 **SELECT** 欄位清單裡的 **CAST(... * ... AS ...)**——沒有任何迴圈，一次 **SELECT** 敘述做完取數 + 篩選 + 計算三件事。

1. **ZR_AM08_DEMO**：

```
``abap REPORT zr_am08_demo.
```

```
PARAMETERS p_carrid TYPE s_carr_id DEFAULT 'LH'. PARAMETERS p_skip AS CHECKBOX DEFAULT abap_true.
```

```
START-OF-SELECTION.
```

```
GET RUN TIME FIELD DATA(lv_t1). DATA(lt_classic) = zcl_am08_revenue_classic=>get_revenues( iv_carrid =
p_carrid iv_skip_unsold = p_skip ). GET RUN TIME FIELD DATA(lv_t2).
```

```
zcl_am08_flight_revenue=>get_revenues( EXPORTING iv_mandt = sy-mandt iv_carrid = p_carrid iv_skip_unsold
= p_skip IMPORTING et_revenue = DATA(lt_amdp) ). GET RUN TIME FIELD DATA(lv_t3).
```

```
DATA(lv_time_classic) = lv_t2 - lv_t1. DATA(lv_time_amdp) = lv_t3 - lv_t2.
```

```
" ...分別加總 lt_classic/lt_amdp 的 revenue 欄位，WRITE 比對筆數/耗時/總營收... ``
```

預期輸出 (實測畫面，多次執行對照)

第一次執行 (AMDP 這支 Method 在這次 session 是第一次被呼叫)：

```
Classic ABAP   : rows = 75   time(us) = 33,394   total revenue = 8,845,505.99
AMDP push-down: rows = 75   time(us) = 208,467   total revenue = 8,845,505.99
```

同一支程式再執行第二、三、四次 (AMDP 的執行計畫已經編譯快取)：

```
AMDP push-down: time(us) = 12,796
AMDP push-down: time(us) = 8,411
AMDP push-down: time(us) = 1,975
```

再跑一次完整比對（兩邊都是「熱」的狀態）：

```
Classic ABAP   : rows = 75   time(us) = 1,565   total revenue = 8,845,505.99
AMDP push-down: rows = 75   time(us) = 3,705   total revenue = 8,845,505.99
```

筆數（75）跟總營收（8,845,505.99）兩個版本從頭到尾完全一致，證明改寫沒有改變邏輯正確性。

團隊實務備註

- 這是這門課最重要的一個效能發現：**AMDP 不是「無條件更快」**——
`ZCL_AM08_FLIGHT_REVENUE=>GET_REVENUES` 第一次呼叫花了 208 毫秒，比 ABAP 版本的 33 毫秒慢了 6 倍多；但接下來幾次呼叫，AMDP 版本掉到 2~13 毫秒，開始跟 ABAP 版本同一個量級、甚至更快。這個現象的合理解釋是：HANA 資料庫第一次執行某個 Database Procedure/Function 時要編譯執行計畫，這個編譯成本是一次性的，編譯完的計畫會被快取，之後同一個 Procedure 再被呼叫就不用重新編譯，直接執行快取好的計畫——這跟很多資料庫/JIT 編譯器「第一次慢、後面快」的行為是同一類現象
- 這個發現直接回答「何時該下推、何時不該」：如果一支報表/API 只會被呼叫一兩次（例如批次跑一次的月結報表），AMDP 的編譯成本可能讓它「感覺」比較慢，這種情境下推的效益不明顯；但如果同一段邏輯會被高頻率重複呼叫（例如一支被前端 API 每秒呼叫好幾次的服務），編譯成本只需要付一次，後面大量次呼叫都吃到「快取好的執行計畫」+「不用把大量原始資料搬到應用層」的雙重好處，這種情境下推的效益才會真正顯現
- 這題的資料量只有 75 筆，本來就小到看不出「搬資料的網路成本」這個 **Code-to-Data** 真正想省的東西——如果 `SFLIGHT/SBOOK` 有幾百萬筆、要先 JOIN 再聚合再篩選，ABAP 版本要把大量原始資料整批搬到應用伺服器才能開始算，這時候 AMDP 版本「資料庫端算完只回傳結果」的優勢才會被放大到有感；這份訓練資料規模完全不夠大到能公平比較兩種寫法在「大資料量」情境下的差異，這題的效能數字只能證明「兩者同量級，AMDP 有暖機成本」，不能拿來當作「AMDP 一定比較快/慢」的普遍結論
- `GET RUN TIME FIELD` 量出來的數字受當下系統負載影響很大，這題連續執行同一支程式，同一個 AMDP 呼叫的耗時就從 208,467 一路降到 1,975，本身就說明這不是嚴謹的效能測試方法（沒有做多次取平均、沒有排除系統其他負載），只適合拿來看「量級差異」「趨勢方向」，不適合拿單一次的數字做嚴謹的效能結論

思考題

1. 如果把 `ZR_AM08_DEMO` 改成連續呼叫 AMDP 版本 100 次、取平均耗時，你預期平均值會比單次測量更接近「暖機後」的耗時，還是更接近「第一次」的耗時？為什麼？多次取平均是比較合理的效能測量方式？
2. 這題的 ABAP 版本用 `LOOP AT ... ASSIGNING` 逐筆算 `revenue`，如果資料筆數從 75 筆變成 7500 萬筆，ABAP 版本要面對哪些改寫前沒想過的問題（提示：不只是「跑比較久」，想想看記憶體、`SELECT` 一次撈幾百萬筆到內表這件事本身合不合理）？AMDP 版本在同樣的資料量級下，會不會也遇到類似問題？
3. `op12` 的重構方向是「取數薄、計算純」（把計算邏輯搬進一個容易單元測試的純函式，但計算本身還是在 ABAP 端跑）；這題的重構方向是「把計算搬進資料庫端」。這兩種重構方向解決的是同一個問題嗎？如果要對 `ZCL_AM08_FLIGHT_REVENUE` 的 AMDP 邏輯寫 ABAP Unit 測試，會遇到什麼跟 `op12` 的純函式測試不一樣的困難？（提示：回想 `am01~am08`，AMDP Method 的測試需要真的連到 HANA 執行，這跟 `op12`「不碰 DB」的純函式測試策略正好相反）

答案

見 `zcl_am08_revenue_classic.clas.abap` (改寫前) 、 `zcl_am08_flight_revenue.clas.abap` (改寫後) 、 `zr_am08_demo.prog.abap` (SAP 端物件 `ZCL_AM08_REVENUE_CLASSIC` / `ZCL_AM08_FLIGHT_REVENUE` / `ZR_AM08_DEMO` , 套件 `$TMP`) 。